

Software Engineering

Chapter 4 - (Software Process and Project Metrics)

Farjana Parvin
Lecturer
Dept. of CSE, RUET



Software metrics refers to a broad range of measurements for computer software.

- Measurement can be applied to the software process with the intent of improving it on a continuous basis.
- Measurement can be used throughout a software project to assist in estimation, quality control, productivity assessment, and project control.
- Measurement can be used by software engineers to help assess the quality of technical work products and to assist in tactical decision making as a project proceeds.

Software Measurement

Direct measures of SE process include cost and effort. Direct measures of product include LOC produced, execution speed, memory size, and defects reported over some set period of time.

Indirect measures of product include functionality, quality, complexity, efficiency, reliability, maintainability, and many other “-abilities”

Size-oriented Metrics

Derived by normalizing quality and/or productivity measures by considering the size of the software that has been produced.

Project	LOC	Effort	\$(000)	Pp. doc.	Errors	Defects	People
alpha	12,100	24	168	365	134	29	3
beta	27,200	62	440	1224	321	86	5
gamma	20,200	43	314	1050	256	64	6
•	•	•	•	•	•		
•	•	•	•	•	•		
•	•	•	•	•	•		

analysis, design, code and test

Before Release

After Release

For example: choose LOC as normalization value.

Size-oriented Metrics

Then we can develop a set of simple size-oriented metrics:

- Errors per KLOC
- Defects per KLOC
- \$ per LOC
- Page of documentation per KLOC

And other interesting metrics can be computed:

- Errors per person-month, LOC per person-month, \$ per page of documentation.

Function-Oriented Metrics

Use a measure of the functionality delivered by the application as a normalization value.

Functionality can not be measured directly, it must be derived indirectly using other direct measures.

A measure called the *function point*.

Function-Oriented Metrics

Function points are derived using an empirical relationship based on countable (direct) measures of software's information domain and assessments of software complexity.

Function points are computed by completing the table shown in the Figure.

Measurement parameter	Count	Weighting factor				
		Simple	Average	Complex		
Number of user inputs		×	3	4	6	=
Number of user outputs		×	4	5	7	=
Number of user inquiries		×	3	4	6	=
Number of files		×	7	10	15	=
Number of external interfaces		×	5	7	10	=
Count total	→					

Function-Oriented Metrics

To compute function points (FP), the following relationship is used:

$$FP = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)]$$

The Fi ($i = 1$ to 14) are "complexity adjustment values" based on responses to the following questions:

1. Does the system require reliable backup and recovery?
2. Are data communications required?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require on-line data entry?
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?
8. Are the master files updated on-line?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

Each of these questions is answered using a scale that ranges from 0 (not important or applicable) to 5 (absolutely essential).

Analyzing the Information Domain

<u>measurement parameter</u>	<u>count</u>	<u>weighting factor</u>					
		simple	avg.	complex			
number of user inputs		X	3	4	6	=	
number of user outputs		X	4	5	7	=	
number of user inquiries		X	3	4	6	=	
number of files		X	7	10	15	=	
number of ext.interfaces		X	5	7	10	=	
count-total							
complexity multiplier							
function points							

Measurement parameter	Count	Weighting Factor					
			Simple	Average	Complex		
Number of user inputs	3	×	3	4	6	=	9
Number of user outputs	2	×	4	5	7	=	8
Number of user inquiries	2	×	3	4	6	=	6
Number of files	1	×	7	10	15	=	7
Number of external interfaces	4	×	5	7	10	=	20
Count total	→						50

•Count total shown must be adjusted using Function Point (FP):

$$\text{FP} = \text{count total} \times [0.65 + 0.01 \times S(\text{Fi})]$$

where count total is the sum of all FP entries and F_i ($i = 1$ to 14) are "complexity adjustment values." For the purposes of this example, we assume that $S(F_i)$ is 46 (a moderately complex product).

$$\text{FP} = 50 \times [0.65 + (0.01 \times 46)] = 56$$

Why Opt for FP Measures?

- ❑ independent of programming language
- ❑ uses readily countable characteristics of the "information domain" of the problem
- ❑ does not "penalize" inventive implementations that require fewer LOC than others
- ❑ makes it easier to accommodate reuse and the trend toward object-oriented approaches

Typical Function-Oriented Metrics

errors per FP

defects per FP

\$ per FP

pages of documentation per FP

FP per person-month